Module 2: Signal Processing

Andrea Moglia, Marco Paracchini

Laboratory of Nearable Technologies for Health Data Science
A.Y. 2025/2026

Agenda

This lesson has the purpose to give you (*and/or refresh*) some **basic signal processing** knowledge that you could exploit during the development of your **project**.

We will skip between **slides** and **code** to (*quickly*) introduce theoretical concepts and immediately apply it.

Some exercises will be done together, for other I will give you about 10/15 minutes to try to solve them by yourself before discussing together the solutions (***It's a lab!***)

Agenda

You can find the code and data we will use on this github.

https://github.com/n-health-lab-polimi/n-health-lab-2026

After the lesson we will push here the code we developed during class

1st task: clone this repository

```
git clone https://github.com/n-health-lab-polimi/n-health-lab-2026.git
```

Agenda

2nd task: create a virtual environment and install the packages you find in ‘requirments.txt’

If you are using Visual Studio Code:

write on the bar: “>Python: Create Environment”

```
pip install -r requirements.txt
```

Agenda

This lesson is dedicated to **signal processing**.

We will have two more similar labs on:

- **Image processing**
- **Machine Learning & Deep Learning**
- **GUI**

Digital Signals

A discrete-time signal is represented by a sequence of values:

$$\{x(n)\,, -\infty < n < \infty\}$$

nth sample of the sequence

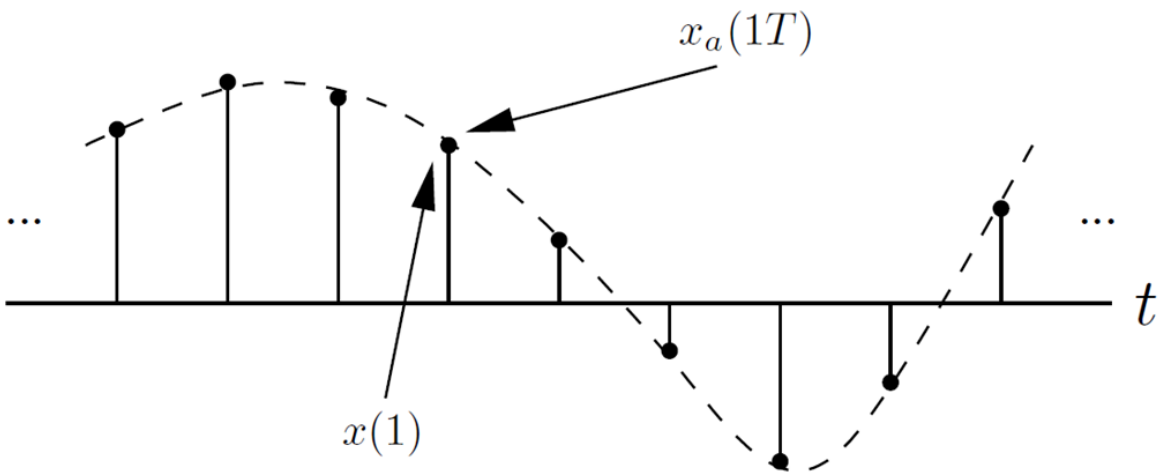
integer

Discrete-time signals are often obtained by sampling continuous-time signals.
In this case, the nth sample is obtained as:

$$x(n) = x_a(nT)$$

Continuous-time signal

Sampling period



Digital Signals

Discrete sequences can be manipulated in several ways. The most basic operations are the following:

- Scaling

$$y(n) = Ax(n) \ , \quad A \in \mathbb{R}$$

- Sum of two sequences:

$$y(n) = x_1(n) + x_2(n)$$

- Product of two sequences:

$$y(n) = x_1(n) \cdot x_2(n)$$

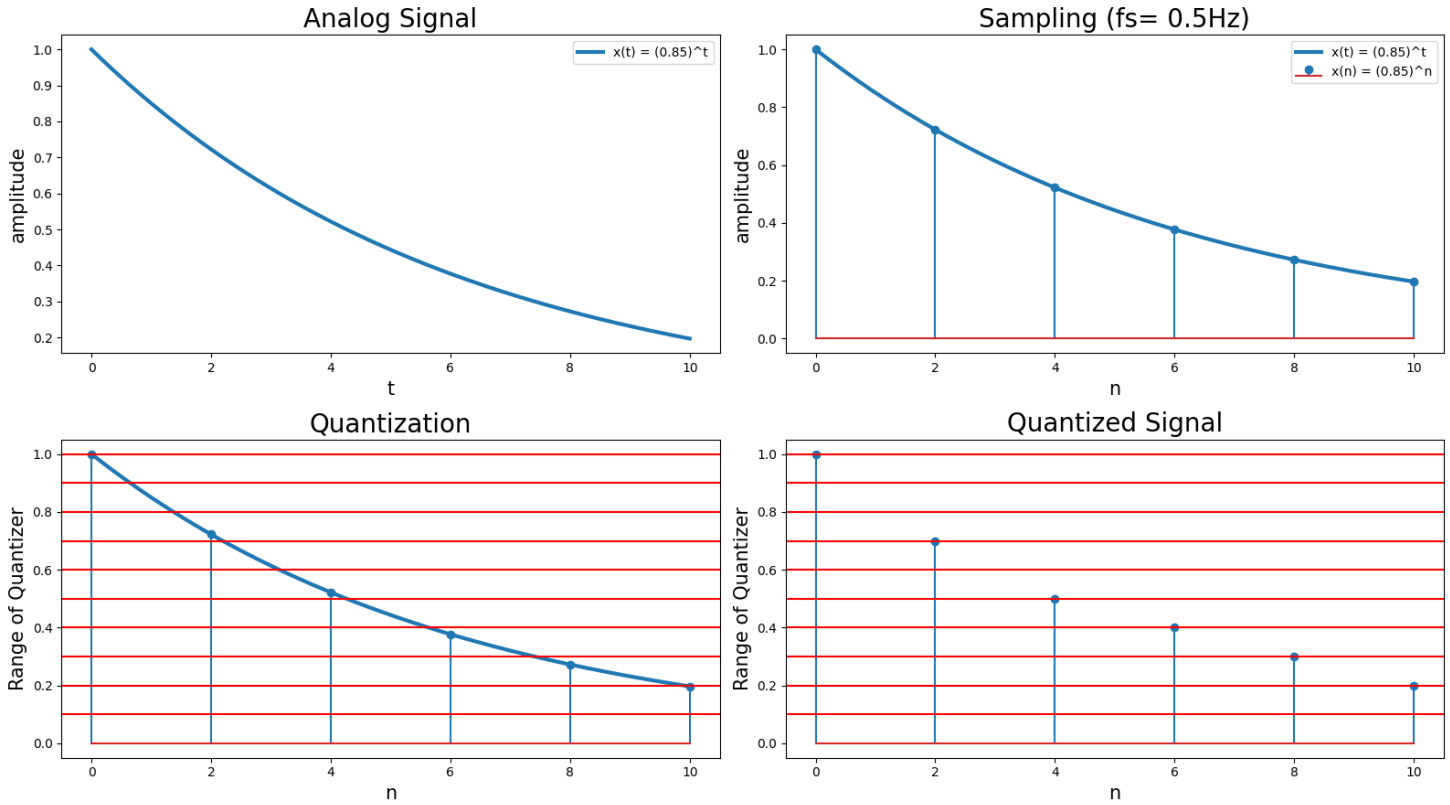
- Time-shifting:

$$y(n) = x(n - n_0)$$

Digital Signals

Additionally in the conversion of continuous time signals a quantization step is applied.

Discrete-time signals are often depicted graphically using stem plots:



Let's switch to python and let's make a graph like this one!

Digital Signals – Ex1

1. Define one second of the following signal:

$$x(n) = \sin(2\pi\, f\, n)$$ f = 5Hz, sampling freq. 1000Hz

2. Find the maximum and minimum value of x(n) and choose the amount of quantization bits (#bits: 2,4,8, etc.) and obtain the number of quantization levels (L=2^#bits)

3. Define the quantization step size using this formula: $\delta = \dfrac{x_{max} - x_{min}}{L - 1}$

4. Apply a uniform quantization $x_q = round\left(\frac{x - x_{min}}{\delta}\right)\delta + x_{min}$

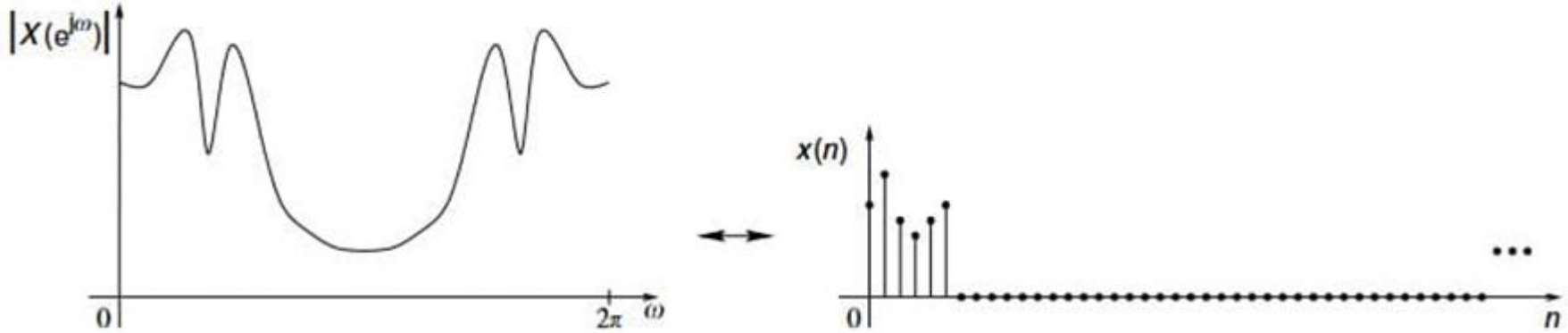
5. Calculate the quantization error as a difference between the quantized and the original signal

6. Plot the two signals

FFT

Signals can be represented in the time domain (as we already did so far) or in the frequency domain:

Frequency domain $X(e^{j\omega})$ Time domain $x(n)$

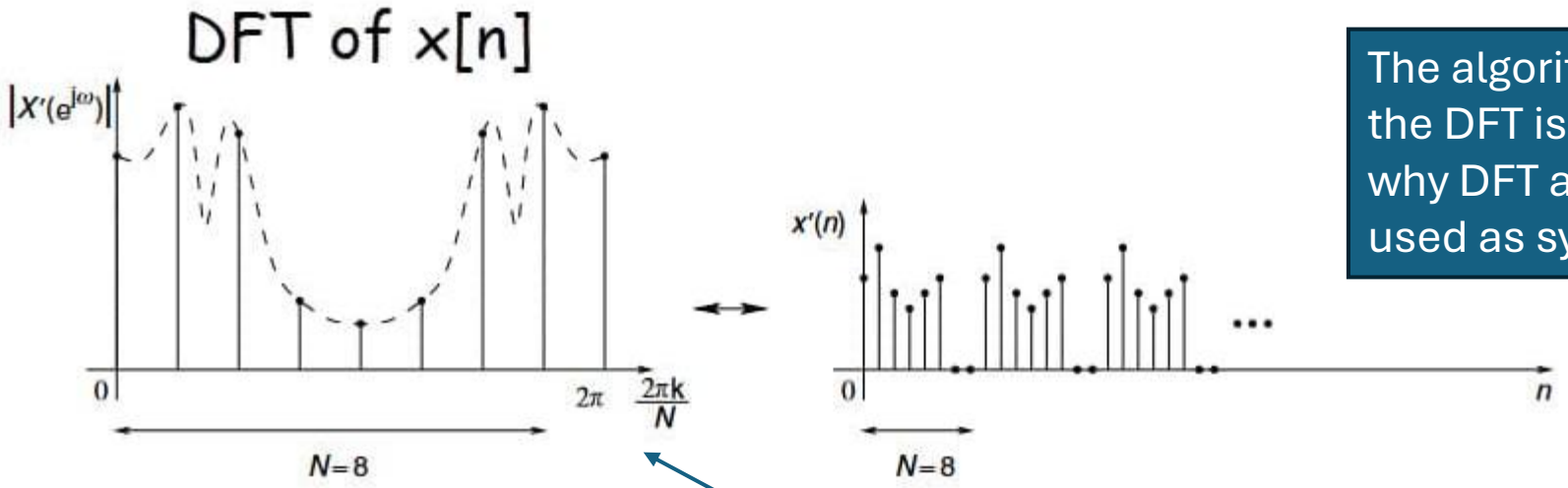


Discrete-Time Fourier Transform (DTFT)

$$X(e^{j\omega}) = \sum_{n=-\infty}^{\infty} x(n)e^{-j\omega n} \qquad x(n) = \frac{1}{2\pi}\int_{-\pi}^{\pi} X(e^{j\omega})e^{j\omega n}d\omega$$

FFT

DTFT output is continuous, what we are really using in practice is the DFT (discrete also in freq. domain)



The algorithm for obtaining the DFT is called FFT (this is why DFT and FFT often are used as synonyms)

Discrete Fourier Transform (DFT)

Limited set of frequencies!

$$X(k) = \sum_{n=0}^{N-1} x(n) e^{-j\frac{2\pi k n}{N}} \qquad x(n) = \frac{1}{N}\sum_{k=0}^{N-1} X(k) e^{j\frac{2\pi k n}{N}}$$

FFT – Ex2

1. Simulate a ppg signal starting from a HR of 60bpm and reaching 120bpm after 20 seconds (Sampling frequency 30 Hz)

2. Compute the FFT and plot the magnitude spectrum

3. Is this the best way to represents the frequency components of the signal?

STFT

In order to apply the DFT to a real signal, we can:

- Calculate the DFT for the whole signal (impractical, especially for long signals and/or realtime applications)
- Split the signal into blocks and calculate a DFT for each block

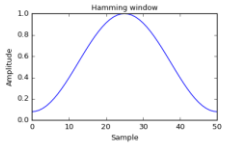
The visualization of the STFT output is called spectrogram

The STFT uses the second approach:

- The signal is split into blocks of equal length
- The blocks are (usually) overlapping
- For each block, the signals is windowed, i.e multiplied with some window functions
- The DFT of the windowed block is computed

The discrete STFT is defined as follows:

$$X(m,k) = \sum_n x(n)w(n-mR)e^{-j\frac{2\pi k}{M}n} \qquad 0 \le k \le M$$



where:
m: is the time frame index
w: is the window function
M: is the length of the window function
R: is the hop size (distance in sample between two consecutive window function)

Let's go back to the previous exercise and use the STFT

Filtering in the freq. domain

Thanks to FFT and IFTT (inverse FFT) we can easily switch between frequency and time domain

Usually noise can be clearly identified in the frequency domain

If we remove it from in the frequency domain and move back in the time domain we filtered the signal!

We can apply filters in the frequency domain using the following steps:

1. Apply the FFT
2. Modify the signal in the frequency domain
3. Apply the IFFT to move back in the time domain

FFT – Ex3

1. Define one second of the following signal:

$$x(n) = \sin(2\pi\, f_1\, n) + \sin(2\pi\, f_2\, n)$$ f1 = 50Hz, f2 = 120Hz, samp. freq. 1000Hz

2. Add noise. Gaussian with 0 avg and 2.5 std.dev.

3. Apply FFT and calculate the PSD (power spectral density, i.e magnitude squared)

4. Define a mask (i.e. binary vector) with the same size of PSD having true values (i.e. 1) if PSD>100

5. Apply the mask setting to 0 the FFT output in locations in which mask = 0

6. Apply IFFT to move back in the time domain

7. Plot the original signal and the filtered one

Convolutions

For two signals $x(n)$ and $h(n)$, the convolution is defined as:

$$(x * h)[n] = \sum_{m=-\infty}^{\infty} x[m]h[n-m]$$

Sliding Window Operation: The filter $h(n)$ slides over $\mathrm{x}(n)$, weighting values to produce a transformed signal.

Convolutions

Convolution theorem

In the **frequency domain**, convolution becomes multiplication:

$$F\{x * h\} = F\{x\} \cdot F\{h\}$$

We can apply a filter directly in the time domain using convolutions!

Convolutions – Ex4

1. Define 1 second of these two signals:

$$x(t) = -5^t \qquad h(t) = \begin{cases} 1, t \geq 0.5 \\ 0, t < 0.5 \end{cases}$$

samp. freq. 100Hz

2. Perform the convolution in the time domain

3. Perform the convolution in the frequency domain

4. See if the convolution theorem holds

Filters

Filters can be represented in the time domain with the following operation

$$y(n) = -a_1 y(n-1) - a_2 y(n-2) - \cdots + b_0 x(n) + b_1 x(n-1) + \cdots$$

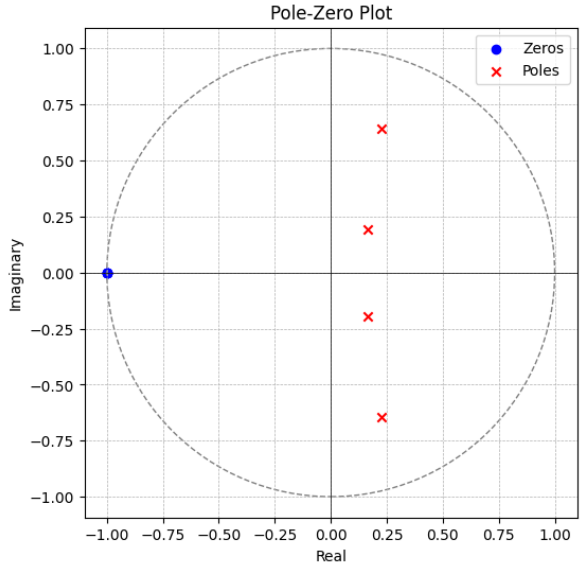
Or in the frequency domain (using the Z-transform, which is a generalization of the DTFT)

$$Y(z) = H(z)X(z) \qquad H(z) = \frac{b_0 + b_1 z^{-1} + b_2 z^{-2} + \cdots}{1 + a_1 z^{-1} + a_2 z^{-2} + \cdots}$$

Poles: Roots of the denominator
Zeros: Roots of the numerator

FIR (finite impulse response) $a_1, \ldots a_n = 0$ (no poles)
Stable but not efficient

IIR (infinite impulse response) (poles and zeros)
Could be unstable but efficient



PPG – Ex5

1. Load the noisy ppg signal stored in ‘data.csv’, sampling freq: 500Hz

2. Apply a band pass filter to remove noisy
   Typical ppg signals are between 30bpm and 180bpm

2. Apply STFT

3. Find maxima in STFT to estimate the heart rate frequency over time

4. Plot the original signal, the filtered one and the HR estimation